

CSE444 Assignment

Vibe Coding Game Development Project Maintenance

Retro Snake Game Maintenance Report

Name: Enis Buğra Okunakol

Course: CSE444

Date: May 2026

1. Introduction

The original project was a Retro Snake Game developed in Python using the PyGame library. The game followed the classic Snake gameplay style, where the player controls a continuously moving snake, collects food to increase the score and length of the snake, and avoids collisions with the walls and the snake's own body. The previous version already included core gameplay features such as keyboard-based movement, random food generation, score tracking, game-over handling, pause/resume functionality, level progression, and persistent high score storage.

This maintenance assignment focuses on evolving the existing Snake game rather than developing a new project from scratch. The main objective was to analyze the current implementation, preserve the original functionality, and extend the system with new gameplay mechanics in a maintainable way. The project was improved by adding multi-level difficulty progression, a robot snake, and special collectable foods that affect the snake's behavior.

The maintenance work was carried out carefully so that the original game structure remained understandable and modular. Instead of rewriting the entire project, the new features were integrated into the existing architecture through small and controlled changes. Existing components such as the game loop, snake movement, food generation, collision detection, and user interface were preserved and extended where necessary. A new robot snake component was added as a separate module to keep the code organized and easier to maintain.

The updated version of the game demonstrates software evolution in practice. It shows how an existing PyGame project can be modified, extended, tested, and documented while maintaining compatibility with its original behavior. This process also reflects the idea of vibe coding, where features are added iteratively, tested immediately, and refined step by step based on the behavior of the working system.

2. Functional Requirements

The updated Retro Snake Game includes both the original game requirements and the newly added maintenance features. The main goal of this maintenance phase was to preserve the existing gameplay while extending the system with new difficulty and interaction mechanics.

The preserved functional requirements from the original version are:

1. The system shall display a grid-based game environment.
2. The system shall allow the player to control the snake using keyboard arrow keys.
3. The system shall move the snake continuously in the current direction.
4. The system shall prevent the snake from directly reversing into itself.
5. The system shall generate normal food at random valid grid positions.
6. The system shall ensure that food does not appear on the snake's body.
7. The system shall increase the snake's length when normal food is consumed.
8. The system shall increase and display the player's score.
9. The system shall detect wall collisions.
10. The system shall detect self-collisions.
11. The system shall provide a game-over state when a collision occurs.
12. The system shall allow the player to restart the game after losing.
13. The system shall provide a start menu before gameplay begins.
14. The system shall allow the player to pause and resume the game.
15. The system shall save and load the high score persistently.

The new functional requirements implemented during the maintenance phase are:

16. The system shall introduce multi-level difficulty as the game progresses.
17. The system shall increase the game speed as the level increases.
18. The system shall calculate the current level based on the player's score.
19. The system shall introduce a robot snake as an additional gameplay object.
20. The system shall allow the player to add a robot snake during gameplay by pressing the B key.
21. The robot snake shall wander randomly around the grid.

22. The robot snake shall not chase food, collectables, or the player.
23. If the player's snake hits the robot snake, the game shall end.
24. If the robot snake hits the player's snake body, the robot snake shall disappear temporarily.
25. The robot snake shall return after 30 seconds when it disappears.
26. The system shall introduce special collectable foods from higher levels.
27. The invincibility food shall make the player temporarily protected from the robot snake.
28. The poison food shall drop part of the snake's tail.
29. The slow-motion food shall temporarily slow down the game.
30. The system shall display temporary effect information, such as invincibility time, slow-motion time, and robot return countdown.
31. The system shall preserve the original Snake game behavior while adding the new maintenance features.

These requirements define the evolved version of the game. The updated system remains compatible with the original gameplay, but it now includes additional challenge, temporary power effects, and a new robot snake mechanic.

3. Approach to the Problem (Vibe Coding)

The maintenance process followed an iterative vibe coding approach. Since the project already had a working version, the first step was not to rewrite the game, but to understand the current codebase and identify where the new features could be added without breaking the original functionality.

The existing project structure was analyzed file by file. The `main.py` file was responsible for starting the game, while `game.py` contained the main game loop, state management, scoring, drawing functions, and collision logic. The `snake.py` file handled the player snake's movement and growth behavior. The `food.py` file handled normal food generation, `ui.py` managed the start, pause, and game-over screens, and `settings.py` stored constants such as screen size, colors, directions, and FPS values.

Before making changes, the original files were backed up. This was important because the assignment was focused on maintaining and evolving an existing system. If a new feature caused an error, the previous working version could still be restored. After that, the maintenance tasks were implemented step by step rather than all at once.

The first change was made in `settings.py`. New constants were added for difficulty progression, robot snake behavior, special food duration, slow-motion timing, invincibility timing, and additional colors. This reduced the need for hard-coded values in the main game logic and made the new features easier to adjust later.

Next, a new `robot_snake.py` file was created. This was a design decision made to preserve modularity. Instead of adding all robot behavior directly into `game.py`, the robot snake was implemented as a separate class. This class manages the robot snake's body, random movement, spawning, and deactivation. Keeping this feature in a separate module made the code easier to read and maintain.

The `food.py` file was then extended to support special collectable foods. The original normal food system was preserved, but a new `SpecialFood` class was added. This class supports three different collectables: invincibility food, poison food, and slow-motion food. Each special food has its own label, color, and effect. This approach avoided changing the original food behavior too much while still allowing the system to support new food types.

The `snake.py` file was modified only minimally. The original movement and growth logic remained the same, but a new `shrink_tail()` method was added. This method is used by poison food to remove part of the snake's tail while keeping the snake playable. This small change allowed the new poison mechanic to be implemented without redesigning the entire snake structure.

Most integration work was completed in `game.py`. The game loop was extended to manage robot snake spawning, robot movement timing, special food spawning, temporary effects, and new collision rules. The game still uses the same main states such as menu, playing, paused, and game over. This preserved compatibility with the original design while adding new gameplay behavior.

Testing was performed after each major change. Syntax checks were run using Python compile commands, and the game was tested manually after integration. The testing process included checking whether the original game still worked, whether the robot snake appeared and moved correctly, whether collision rules behaved as expected, and whether special foods applied their effects correctly.

The main challenge during the maintenance process was integrating new behavior into an already working game without damaging existing features. For example, the robot snake required new collision logic, but the original wall collision and self-collision rules still had to work normally. Similarly, slow-motion and invincibility effects required time-based logic, so `pygame.time.get_ticks()` was used to manage temporary states. By implementing and testing each feature in small steps, the project was extended safely and gradually.

4. Implementation Overview

The updated Retro Snake Game is implemented in Python using the PyGame library. The original modular structure was preserved, but several files were extended and one new module was added to support the maintenance features. The implementation still follows a game-loop structure where the program repeatedly handles input, updates the game state, checks collisions, and renders the screen.

The main entry point of the project is `main.py`. This file creates a Game object and starts the game loop. Since the original entry point was already simple and functional, no major changes were needed in this file.

The `game.py` file remains the central controller of the project. It manages the main game loop, game states, user input, score, level, collision rules, robot snake behavior, special food effects, and rendering. The original game states such as MENU, PLAYING, PAUSED, and GAME_OVER were preserved. New behavior was added inside the existing structure instead of replacing the original flow. This helped maintain compatibility with the previous version.

The difficulty system was extended through score-based level progression. The level is calculated according to the player's score, and the game speed increases as the level increases. This provides multi-level difficulty without requiring a separate level-selection screen. Constants such as initial FPS, maximum FPS, foods per level, and special feature levels are stored in `settings.py`.

A new file named `robot_snake.py` was added for the robot snake feature. The robot snake is represented as a separate class with its own body, direction, spawn logic, movement logic, and deactivation behavior. The robot snake moves randomly around the grid and does not chase food or the player. If the player's snake hits the robot snake under normal conditions, the game ends. If the robot snake hits the player's snake body, the robot snake disappears and returns after 30 seconds. The player can also add a robot snake manually during gameplay by pressing the B key.

The `food.py` file was extended to support both normal food and special collectable foods. The original Food class was kept for normal food behavior. A new SpecialFood class was added for three collectable types: invincibility, poison, and slow-motion. Each special food has a different label, color, and gameplay effect. Invincibility temporarily protects the player from the robot snake, poison removes part of the snake's tail, and slow-motion temporarily reduces the game speed.

The `snake.py` file still manages the player snake's movement, direction changes, growth, and self-collision detection. A small additional method, `shrink_tail()`, was added for the

poison food effect. This method removes part of the snake's tail while keeping a minimum snake length so that the game remains playable.

The `ui.py` file was updated to display the new control instruction for adding the robot snake. The heads-up display in `game.py` was also extended to show temporary status information such as robot return countdown, slow-motion time, and invincibility time. These indicators help the player understand the current active effects during gameplay.

The `settings.py` file stores constants for screen size, grid size, colors, directions, game speed, robot snake timing, and special food timing. Moving these values into settings improves readability and makes future adjustments easier.

Overall, the implementation preserves the original gameplay architecture while extending it with additional interactive mechanics. The updated project remains modular because the new robot snake and special food behavior are separated into clear components instead of being mixed entirely into one file.

5. Discussions and Reflections

The maintenance process showed that extending an existing software project requires a different mindset than developing a new project from the beginning. In the original take-home project, the main focus was to create a working Snake game. In this maintenance assignment, the main focus was to understand the existing implementation, preserve the working features, and add new behavior without damaging the original system.

One of the most important technical changes was the addition of the robot snake. This feature introduced a new moving object into the game world. The robot snake had to move independently from the player, avoid being controlled by the user, and follow its own random movement logic. At the same time, it had to interact with the player's snake through collision rules. If the player hits the robot snake, the game ends. If the robot snake hits the player's snake body, the robot snake disappears for 30 seconds. This required careful integration into the existing collision detection system.

Another major change was the special food system. The original game only had normal food, which increased the score and length of the snake. The updated version adds three collectable food types: invincibility, poison, and slow-motion. These foods affect the game in different ways. Invincibility temporarily protects the player from the robot snake, poison removes part of the snake's tail, and slow-motion temporarily reduces the game speed. This made the gameplay more dynamic and required new time-based logic.

A challenge during implementation was managing temporary effects. Invincibility, slow-motion, and robot disappearance all last for a limited time. To solve this, the game uses PyGame's timing mechanism through `pygame.time.get_ticks()`. This made it possible to

check how much time had passed and display countdown information on the screen. This approach also avoided blocking the game loop, so the game continued running smoothly while effects were active.

Another challenge was preserving compatibility with the original game. The existing features such as movement, food consumption, scoring, pause/resume, game-over state, restart, and high score persistence had to continue working normally. Because of this, the changes were made incrementally. Instead of rewriting the entire project, new behavior was added around the existing structure. This reduced the risk of breaking the original functionality.

The project also demonstrated the importance of modularity in software maintenance. Adding `robot_snake.py` as a separate module made the robot snake easier to manage and kept the main game file from becoming too confusing. Similarly, extending `food.py` with a `SpecialFood` class kept normal food and collectable food behavior separated. These decisions improved readability and made the system more extensible.

Testing was an important part of the maintenance process. After each major modification, syntax checks were performed and the game was manually tested. The final testing process included checking the original gameplay, level progression, robot snake behavior, robot collision rules, special food effects, pause/resume behavior, restart functionality, and high score persistence. This step-by-step testing approach helped identify problems early and made the final version more stable.

From this assignment, the main lesson learned is that software maintenance is not only about adding new features. It also requires understanding the original design, making careful changes, avoiding unnecessary rewrites, and keeping the system readable for future modifications. A working system can become harder to maintain if new features are added without structure. Therefore, separating responsibilities into clear modules is important for long-term adaptability.

In the future, the game could be improved further by adding sound effects, a difficulty selection menu, visual animations, different themes, and more advanced robot movement. The robot snake could also be improved with smarter obstacle avoidance or multiple robot snakes at higher levels. Another possible improvement would be adding a better settings menu where the player can enable or disable features before starting the game.

During the maintenance process, AI-based assistance was used to support code analysis, feature planning, debugging, and documentation. However, the implementation was tested manually, and the final decisions were made based on whether the updated game preserved the original behavior and satisfied the required maintenance tasks.

Overall, the updated Retro Snake Game successfully preserves the original gameplay while adding meaningful maintenance features. The project evolved from a basic Snake game into a more interactive and challenging version with multi-level difficulty, robot snake behavior, and special collectable foods.